System Verilog For Verification

In order to verify that hardware wrriten in HDL (Hardware Description language) is correct,\
there is a need for a language with more features (like OOP) that will support complicated testing procedures.\
System Verilog For Verification is an extension of verilog with many verification features that allow us to build complex simulations.

Doing the exercise

This exercise is split into parts, each covers some helpful skills for a verification developer.\
Each part includes an `README.md` document with instructions and guidance for that part, and a link to the next part.\
This is a simple and quick exercise. The main purpose is to experience coding System-Verilog and get to know some cool features.

Here are the the parts relevant to sector training:

Part 0 - Data Structures (Start Here!)\
Part 1 - Loops\
Part 2 - Tasks And Functions\
Part 3 - Streaming Operations\
Part 4 - Randomization

In each part, you can use all knowledge you have learn until that point.\
Please print the results of each part. It will help the trainers to check your code :)

Congrats! Doing those parts means you started your Team Training!
Here we will go deeper and learn some new cool features SV allow us.

Before starting the exercises below, please indulge us by answering the following questions:

What is the difference between pointer and handle?
Find out who is Shriki?
Tell your instructor the answer (^-^)

Please proceed to the following parts:

Part 5 - Processes\
Part 6 - Events\
Part 7 - Assertions

Most of the exercise consists of self study.\
Everything that you need to know can be found on-line, or here:

Prerequisites

To do this exercise, you need to know basic programing concepts.

Sounds simple, doesn't it? Get started and have some fun :)

next ->

Part 0 - Data Structures

<- prev | *next ->*

Popular Data Structures

Since verification of hardware can become more complex and demanding, data types in Verilog are not sufficient to develop efficient test benches.\
Hence, SystemVerilog has extended Verilog by adding more C like data-types including different types of arrays.

Short explanation about those types below:

Static Arrays:

An array whose size is known before compilation time and is constant.

Dynamic Arrays:

An array whose size in unknown before compilation time. It's size can be set or changed during run time.

Associative Arrays:

An associative array contains pairs of key and value. The content is stored with a certain key, array[key] = value.

Queues:

A queue is a data type where data can be either pushed into the queue or popped from the queue. Implemented by FIFO methodology.

Structures:

A structure can contain elements of different data types which can be referenced as a whole or individually by their names. This is quite different from arrays where the elements are of the same data-type.

Exercises:

Please print the result of each exercise.\
It will help the trainers to check your code :)

Exercise 1:

Create a struct to represents a trainee.\
The struct's fields should defined by different types including int, string, unpacked array, struct...\
At least 4 fields defined by different types.\
Be creative :)

Create an associative array that stores elements of your struct type. Each element should represent one of you.\
Choose key type that will allow us to access each element by the trainee's name.\
Print the array values and keys.

Exercise 2:

Create a new queue that stores the trainees birthday dates from the oldest to the youngest one.\
Find way to represent it without using string's array :)\
Using one line, print the last element in queue.\
Implement the print using two different ways to accses the data memebers.\

<- prev | *next ->*

Part 1 - Loops

<- prev | *next ->*

Loops

A loop is a piece of code that keeps executing over and over.\
System verilog offers different looping constructs.

Short explanation about those constructs below:

While loop:

It will repeat the block as long as the condition is true.

Do-while loop:

This executes the code first and then checks for the condition to see if the code should be executed again.

Foreach loop:

Used mainly to iterate through all elements in an array, because you don't have to find the array size.

For loop:

It will repeat the block until the value will reach it's destination. This allows you to mention starting value, condition and incremental expression all on the same line.

Forever loop:

This is an infinite loop, just like while (1).

Repeat loop:

Used to repeat statements in a block a certain number of times.

Exercises:

Please print the result of each exercise.\
It will help the trainers to check your code :)

Exercise 1:

The code you got defines a long integers queue.\
Remove elements from the queue until the remained queue will contain 4 elements.

Exercise 2:

Create an integers array which contains 200 cells.\
Define a parameter of the array size (Attention: parameters can not be declared inside procedural blocks).\
Initialize the odd cells to zero and the even cells to contain their own index using for loop.

Exercise 3:

Create a two dimensional array that stores multiplication board values using foreach loop.

<- prev | *next ->*

Part 2 - Tasks And Functions

<- prev | *next ->*

Tasks And Functions

Often times we find certain pieces of code to be repetitive and called multiple times.\
In such cases, we can declare a function or task and place the repetitive code inside.

So, what is the differences between function and tasks?

Tasks:

A task doesn't have returned value. like value it can calculate multiple result values and return them using output arguments.\
The main difference between the two is that tasks can contain simulation time consuming elements.

Functions:

The primary purpose of a function is to return a value that can be used in an expression.\
It can "return" multiple outputs using passing arguments by reference (read about it) or using output arguments.\
Functions cannot advance simulation time. Therefor, function cannot conatain tasks that contain simulation time consuming elements.

Arguments Passing:

Arguments can be passed to function or task by value or by reference.\
Read about it and understand what is the difference between the two.

Exercises:

Please print the result of each exercise.\
It will help the trainers to check your code :)

Exercise 1:

The code contains 2 integer arrays.\
Multiply all cells in each array and compare the results.\
Display the array with the bigger result.

Exercise 2:

Write a function or a task that will add delay of 10ns.\
Find a way to approve it works properly :)

Write function or task that displays the last value of a given queue every 10ns until it is empty.

Exercise 3:

Write a function or task that edit a given dynamic array such that each cell contains the sum of all cells with smaller or equal index. \
Do not use any other data structure.

<- prev | *next ->*

Part 3 - Streaming Operations

<- prev | *next ->*

Streaming Operations

The verification of digital circuits requires dealing with bits and bytes.\
It is not a trivial thing to pack or unpack bits, bytes, half words, words, of user defined data structures.

And so, we use streaming operations to apply a variety of manipulations to our data structures in order to pack (or unpack) it into other data structure.

Exercises:

Please print the result of each exercise.\
It will help the trainers to check your code :)

Exercise 1:

Write a function that gets byte as input.\
The function reverses the order of pairs of bits in the given byte and return the modified byte.\
ATTENTION: do not declare variable and use streaming operators in the same coding line (one-liners).

Exercise 2:

Do the following instructions:

1. Use the given bits array.
2. Swap between the first half of the array (The MSB) and the last half (The LSB).
3. Within each byte, reverse the order of its bits.
4. Print the queue in Hex form.

Did you succeed? you are BADASS!!

<- prev | *next ->*

Part 4 - Randomization

<- prev | *next ->*

Randomization

Complex designs have a lot of scenarios and many corner cases that are better verified by randomized tests.\
Randomization helps us achieve different scenarios for every run. This will ensure that we hit corner cases and uncover any hidden bugs.\
SystemVerilog allows users to specify constraints that processed by an internal solver to generate random values that satisfy all conditions.\
Constraints are a way to let us define what legal values should be assigned to the random variables.

Short explanation about those constraints below:

Constraint on expression:

Constraints can be placed on variables as well as on expressions.

```
localparam int DATA_WIDTH = 4;
rand int num;
constraint num_c {

    // Num will be randomized in a way so that it could be devided by DATA_WIDTH
without a remainder.
    num % DATA_WIDTH == 0;
}
```

Inside Constraint:

The inside keyword in SystemVerilog allows to check if a given value lies within the range specified using the inside phrase.

```
rand int m_var;
constraint var_c {

    // The Value of m_var will be inside the range of 0 to 10.
    m_var inside {[0:10]};
}
```

Foreach Constraint:

SystemVerilog provides the support to use foreach loop inside a constraint so that arrays can be constrained. The foreach construct iterates over the elements of an array or queue.

```
rand int m_q [$];
constraint queue_c {

    // The queue m_q will be randomized so each member of the queue will have the
value of his index.
    foreach (m_q[i]) {
        m_q[i] == i;
    }
}
```

Distribution Constraint:

SystemVerilog provides a way to create a weighted distributions using the `dist` operator.\
The `dist` operator can be used in conjunction with the `:=` operator to split the weight equally for every specified value in the range or with the `:/` operator to split the weight equally between all the values.

```
rand int num1;
rand int num2;
constraint num1_c {

    // In this example the num_1 variable has weight of 10 to be 0, 20 to be 7,
10 to be 8 and every number in the range of 1 to 6 has a weight of 10, so in
total sum of weight is 100 .
    num1 dist {
        0 := 10,
        [1:6] := 10,
        7 := 20,
        8 := 10
    };

    // In this example the num_2 variable has weight of 10 to be 0, 20 to be 7,
10 to be 8 and 10 to be a number in the range of 1 to 6 (i.e. 10/6 for every
number in the range), so the total sum of weight is 50.
    num_2 dist {
        0 :/ 10,
        [1:6] :/ 10,
        7 :/ 20,
        8 :/ 10
    };
}
```

Randomization Methods:

Variables that are declared as rand or randc inside a class are randomized using the built-in randomize() method.\
The method returns 1 if randomization was successful, and 0 if it failed.\
Class objects are not randomized automatically, and hence we should always call the randomize() method to do randomization.\
randomize() method is built-in and cannot be overriden.

Multiple Constraints

There can be placed multiple constraints on the same expression/variable while the constraints do not contradict one another.

```
byte payload [$];
constraint payload_c {

    // The payload size has a probability of 30 to be under 100 and a probability
of 70 to be between 101 and 200.
    payload.size() dist {
        [1 : 100] :/ 30,
        [101 : 200] :/ 70
    };

    // The payload size have a 50 50 chance to be an odd or an even number.
    payload.size() % 2 == 0 dist {
        1'b0 :/ 50,
        1'b1 : 50
    };
}
```

Short Questions:

1. In order to randomize local variables std::randomize() and .randomize() can be used. What is the difference between the two?
2. What unique does?

Exercises:

Please print the result of each exercise.\
It will help the trainers to check your code :)

Note: Be aware that constraints has to be written inside class. Read about randomization local variables with constrains.

Exercise 1:

Create an integers array contains between 20 to 40 cells. Randomize the array's values such that:\
The odd cells contains numbers in range 10 to 100.\
The even cells numbers according to the following distribution : \
1/3 for range 0 - 9.\
2/3 bigger than 100.\
Additionaly, the even cells should have a 30% chance to be divisible by 43 without remainder.

Exercise 2:

Trip day!, the teacher wants to give a different number to each one of the 12 students (not including 0).\
Create an associative array that will represent the class students and their given number.

<- prev | *next ->*

Part 5 - Processes

<- prev | *next ->*

Congrats! Doing this part means you started your Team Training!
Make sure you answered the questions in the main README page.

Processes

A thread or process is any piece of code that gets executed as a separate entity.\
In verilog, each of the initial and always blocks are spawned off as separate threads that start to run in parallel from zero time.\
SystemVerilog provides support for parallel or concurrent threads through fork join construct.\
Multiple procedural blocks can be spawned off at the same time using fork and join.

There are variations to fork join that allow the main thread to continue executing rest of the statements based on when child threads finish.

Fork Join:

A fork and join will allow the main thread to resume execution of further statements that lie after the fork, if all of the child threads complete.\
If five threads are launched, the main thread will resume execution only when all of the five threads finish execution.

Fork Join_Any:

A fork and join_any will allow the main thread to resume execution of further statements that lie after the fork, if any one of the child threads complete. If five threads are launched, the main thread will resume execution only when any one of the five threads finish execution.\
Once the main thread resumes operation, the remaining four threads that were launched will continue to run in the background.

Fork Join_None:

A fork and join_none will allow the main thread to resume execution of further statements that lie after the fork regardless of whether the forked threads finish. If five threads are launched, the main thread will resume execution immediately while all the five threads remain running in the background.



Exercises:

Please print the result of each exercise.\
It will help the trainers to check your code :)

Exercise 1:

Lets create a game.\
There are two competitors.
Each competitor wait a randomized delay and than randomize an integer between 100 to 1000. \
Repeat this five times and sum all integers you got For each player. \
At the end of the game, print the winner name.

Exercise 2:

Create a task that randomizes a delay and waits for that same delay.\
Run the task three times in parallel and display the smallest delay as soon as it ends.

<- prev | *next ->*

Part 6 - Events

<- prev | *next ->*

Events

An event is a static object handle to synchronize between two or more concurrently active processes. One process will trigger the event, and another process waits for the event.

Wait:

The wait function halts the code from proceeding until a given expression occurs.

@ Operator:

The @ operator, in contrast to the wait function, waits only for an event to be triggered from when the time the code reaches the operator and onwards.

.triggered property:

The triggered property is a property of an event which is equal to True when an event has occurred in the the current simulation time.

Exercises:

Please print the result of each exercise.\
It will help the trainers to check your code :)

Exercise 1:

There are elections for the best song of 2022.\
Fill the two functions or tasks within the fork, one will illustrate the voting process, randomize vote and wait randomized delay until the next vote.\
The other will print the time a vote was added.\
Implement the vote_timing using two different ways. Once with wait statements and once with @.\
Use an events to synchronize between the processes.

<- prev | *next ->*

Part 7 - Assertions

<- prev

Assertions

Assertions are tools that allow us to make sure certain properties of a module's logic always happen.

Assert:

Asserts whether a boolean expression is true or false.

Sequences:

A sequence (not to be confues with a UVM sequence), is a boolean expression that defines a series of signal values.
Some examples:

```
A ##5 B      - A should happen now, and in 5 clocks B should happen.
A ##[1:5] B  - A should happen now, B should happen in the next 1 to 5 clocks.
A ##5 B [*3] - A should happen now, B should happen every 5 clocks, three times.
A ##[2:$] B  - A should happen now, B should happen between 2 clocks and
simulation finish.
```

Implication operators:

Used to create a correlation between sequences. If Sequence 1 happens then Sequence 2 should happen.

```
A |-> B - If A happens B should happen on the same clock.
A |=> B - If A happens B should happen on the next clock.
```

Assertion Functions:

Functions used to describe an activity of a signal.

```
$rose(A)    - When A was 0 last clock and is 1 now.
$fell(A)    - When A was 1 last clock and is 0 now.
$stable(A)  - When A stays the same value.
$past(A,x)  - Returns the value of A, x clocks in the past.
```

Combining Sequences:

```
A and B         - A and B should always be true.
A or B          - Either A or B should be true at all times.
A intersect B   - A and B should always be true and end at the same time.
A within B      - A should happen once within B.
A throughout B  - A should be happen at all times while B is happening.
```

Disabling Sequences:

```
disable iff (x) - Disables a property if x is true, when an assertion is disabled
it is not checked.
```

For more information checkout System Verilog Book.

Or Chip Verify Website.

Exercises:

For each excercise create an assertion which verifies the following:

Exercise 1:

5 clocks after signal 'a' is high for 3 clocks in a row, signal 'b' will be low.

Exercise 2:

While signal 'c' is high, signal 'd' should be high at least once.

Exercise 3:

Signals 'e' and 'f' raise together, signal 'f' lowers before signal 'e'. Only while signal 'g' is high.

Exercise 4:

The signal 'i' is always equal to the value of signal 'h' 6 clocks ago.

<- prev